

Design and Implementation of a DSL for Django Backends

Introduction

Building web backends with frameworks like Django often involves writing repetitive boilerplate code for models, relationships, and REST endpoints. A dedicated domain-specific language (DSL) can raise the level of abstraction and let developers specify the application structure in a concise, readable form. DSLs “allow for raising the level of abstraction, improving development productivity, and establishing an equitable communication between domain experts and developers”[1]. By focusing on common patterns (e.g. model definitions and CRUD APIs), the DSL becomes easier to use for non-programmers or newcomers. Once defined, the DSL compiler can generate Django code automatically. This code generation approach yields significant benefits: a single high-level description can produce entire application code, saving time and reducing errors[2][1]. In summary, a custom DSL for backend development can improve productivity and consistency by automating boilerplate and ensuring uniform patterns across models and endpoints.

DSL Overview

The DSL is designed to let users declare **models**, **fields**, **relationships**, **enumerations**, and **API endpoints** in a simple syntax. Key features include:

- **Model Declarations:** Users define a model with fields and types. For example:

```
model User {  
    name: String ;  
    age: Integer ;  
    status: Role;  
}
```

This declares a Django User model with fields name (string), age (integer), and status (an enum).

- **Field Types & Relationships:** Basic field types (e.g. String, Integer, Boolean) are supported. If a field’s type matches another model name, it is treated as a relationship (a foreign key). For example, `author: User;` in a Book model would generate a `ForeignKey(User, ...)`.
- **Data Validation :**
 - Integer
 - Min

- Max
- **Dates and Time**
 - Min
 - Max
- **Wildpattern for Strings**
 - the official regex with all its options including *, +, ., (), [], ?, and...
- **Annotations:** Fields can have annotations (prefixed with @) to specify constraints or metadata. Examples include @pk (primary key), @unique, @nullabe , @valid[...]. These map to Django field options like primary_key=True or unique=True. For instance:

```
model Profile {
  id: Integer @pk @valid[min=8 , max=10];;
  username: String @unique @valid[wildpattern='a+'];;
}
```

- **Enumerations (enums):** Named sets of values can be declared with enum. For example, enum Role { ADMIN, USER, GUEST } defines a Python Enum or Django TextChoices that can be used as a field type (status: Role;).
- **API Endpoints:** The DSL allows declaring REST endpoints by HTTP method and path, with optional input/output types. Example:

```
endpoint listUsers: GET /users {
  response: User;
}

endpoint condintional_Users: GET /users/{x}/{y} {
  response: relational {

    r_1 = Select<name eq x, age eq y>(User);

    r_2 = Project<name, last_name>(r_1);

    r_3 = Select<>(Person);

    r_4 = Join_inneer<name>( r_2 , r_3 ) ;

    -> r_4 ;
  }
}
```

```
};

}
endpoint createUser: POST /users {
  input: "User_Json_String" ;
  response: User;
}
```

Here, `listUsers` is a GET endpoint returning a list of `User` objects, and `createUser` is a POST endpoint taking a `User` payload and returning a created `User`. This high-level endpoint declaration simplifies setting up corresponding Django views and URL routes.

all operations in block statements :

```
Select<key operator value,...>(relation)
Project<key,...>( relation)
Join_[type]<key,...>( relation , relation)
    type = [inner|outter|right|left]
Union( relation , relation)
Intersection( relation , relation)
Difference( relation , relation)
Cartesian( relation , relation)
Len(relation)
Limit<start,stop,step>(relation)
```

Operands may be a relation (table) or temporary variables saved in the block.

- **Comment:** In your DSL, you can add comments that are used solely for display in your code and have no executable functionality in the target generated code.

For this purpose, you can:

- Write single-line comments using `\\ ...`
- Write multi-line comments using `""" ... """`

In short, the DSL is a concise language for describing the data model and API of a Django backend. By restricting the language to these domain concepts, it ensures the generated code is consistent and closely matches the user's intent.

Compiler Architecture

The compiler architecture is designed to be extensible for multiple target frameworks. While the current implementation generates Django code, the AST structure is framework-agnostic. Developers can implement their own code generation modules for other frameworks (like Flask, FastAPI, etc.) by traversing the same AST and emitting appropriate code for their target platform.

The DSL compiler is structured in classical front-end/back-end phases:

- **Lexer:** Using ANTLR, we define tokens (like NAME, NUMBER, punctuation) and keywords (model, enum, endpoint, HTTP methods, etc.). The lexer scans the input DSL text and produces a stream of tokens. Lexer rules skip whitespace and comments, and recognize annotations (e.g. @unique) and symbols ({, }, ;, etc.).
- **Parser and Parse Tree:** The ANTLR-generated parser processes the token stream according to the grammar. It constructs a parse tree where each node corresponds to a grammar rule (e.g. a modelDecl node for each model). This tree captures the exact structure of the source DSL. By using ANTLR, we rely on a robust parser generator: the grammar formally describes our DSL, and ANTLR automatically handles the details of parsing[3]. If there are syntax errors, ANTLR reports them at parse time.
- **Custom Listener:** We generate ANTLR listener classes from the grammar (.g4) file. A *listener* includes “enter” and “exit” methods for each rule[4]. We implement a subclass (e.g. DjangoDslListener) that overrides relevant methods. When the parser walks the parse tree, it calls our enterModelDecl, exitFieldDecl, etc. methods. Inside these methods, we build an internal representation (AST) of the DSL constructs. For example, when enterModelDecl is called, we create a new Model object; when exiting a field declaration, we add a Field child to the current model. This listener-based approach cleanly separates parsing from semantics: the grammar remains a pure syntax specification, while our listener contains the logic to assemble model definitions.
- **AST Structure:** The compiler constructs an Abstract Syntax Tree (or similar object model) representing the DSL program. For instance, we may have classes like DslModel, DslField, DslEnum, and DslEndpoint. A DslModel would hold its name and a list of DslField objects, each with type and annotations. A DslEndpoint would hold its HTTP method, path, and request/response types. This AST simplifies code generation: instead of navigating the raw parse tree, code generators work with a more convenient structure. Converting the parse tree to an AST often involves walking the tree (depth-first) and instantiating objects. (Tools like visitors or manual tree walks can also create AST nodes suited to the application, as discussed in the literature[5].)

- **Code Generation Modules:** Finally, the compiler traverses the AST and emits Django code. We organize generation into modules corresponding to output files:
- **models.py Generator:** This module iterates over each `DslModel` and generates a Django model class. For example, a User model with fields `id`, `name`, `status` would produce Python code like:

```
from django.db import models

class User(models.Model):
    id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=255)
    status = models.CharField(choices=[(Role.ADMIN, 'ADMIN'),
    (Role.USER, 'USER')], max_length=10)
```

If an annotation like `@pk` is present, we set `primary_key=True`. For enum fields, we generate a corresponding Python Enum class (or Django TextChoices) named `Role` and use it in the model.

- **views.py Generator:** This module creates Django view functions or classes for each endpoint. For a simple GET endpoint, it might generate a view that queries the database and returns JSON. For example, `listUsers` might generate:

```
from django.http import JsonResponse
from .models import User

def listUsers(request):
    users = list(User.objects.values())
    return JsonResponse(users, safe=False)
```

For POST endpoints, we generate logic to create objects from JSON. The generator fills in templates or builds strings based on AST data (model names, fields, etc.) to produce valid Python code.

- **urls.py Generator:** This module creates Django URL routes that connect HTTP paths to the views. For example:

```
from django.urls import path
from . import views

urlpatterns = [
    path('users/', views.listUsers, name='listUsers'),
    path('users/create/', views.createUser, name='createUser'),
]
```

The paths and view names come from the `endpointDecl` AST nodes.

In all generation steps, template strings or print statements include the correct imports and boilerplate. For instance, each generated `models.py` begins with `from django.db import`

models. The Strumenta guide notes that once we have an AST, code generation can proceed by simple tree traversal and string output[3]. Using this approach, the entire Django backend code (models, views, URLs) is produced automatically from the DSL specification.

Example

To use it, you can run the `main.py` file with appropriate arguments via the *terminal*.

```
usage: main.py [-h] [--version] [--generate] [--astimg] -i INPUT [-o OUTPUT] [--baseinput BASEINPUT] [--baseoutput BASEOUTPUT] [--target TARGET]

DSL to Backend code generator

options:
  -h, --help            show this help message and exit
  --version             show program's version number and exit
  --generate            Enable Code Generator (default: True)
  --astimg             Build and Show AST image (default: False)
  -i, --input INPUT    Input file name (default: None)
  -o, --output OUTPUT  Output directory (default: generated_app)
  --baseinput BASEINPUT
                        Base directory for input file (default: .)
  --baseoutput BASEOUTPUT
                        Base directory for output file (default: .)
  --target TARGET      Target framework for code generation from AST tree (default: Django)
```

For Exm

```
python main.py --baseinput test_grammer_files -i test_endpoint_1.txt --astimg
test_grammer_files/ test_endpoint_1.txt
```

```
model User {
  username : String @pk @non-nullable @unique @valid[wildpattern="..."];
  age : Int @nullable @valid[min=8,max=14];
  bth : Date @valid[min="2020-01-01" , max="2024-06-11"];
}

endpoint listUsers : GET "/users" {
  response : User ;
}

endpoint first_User : GET "/users/{x}/{y}" {
  response : relational {
    r_1 = Select<name eq y,age lst x> (User);
    r_2 = Project<name,lastname>(r_1);
    r_3 = Select<>(Person);
    r_4 = Join_inner<name,username>(r_2,r_4);
    r_5 = Project<name,lastname,bth>(r_4);
    len_temp = Len(r_5) ;
    r_6 = Limit<1,len_temp,3>(r_5);
    -> r_6 ;
  };
}

endpoint sign_in : POST "/users/sign-in" {
  input : "
  {
    name : 'mamad' ,
```

```

    age : 18 ,
    sex : 'Male'

}
"; // json string without grammar checking (if it is not in form of json
that must error in runtime!)
    response : User;
}

```

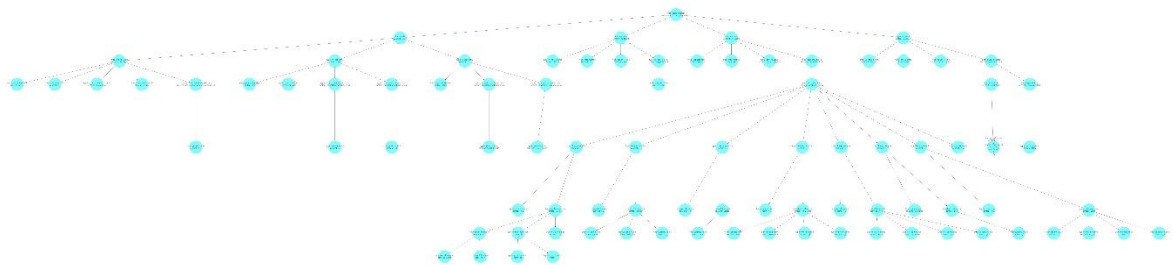
Parse Tree

For a clearer picture, refer to the other source files inside the zip file.



AST Tree

For a clearer picture, refer to the other source files inside the zip file.



Example Generated Code (Django Target)

In generated_app directory we have :

Admin.py

```

from django.contrib import admin
from .models import *
admin.site.register(User)

```

Model.py

```

from django.db import models
from django.core.validators import MinValueValidator, MaxValueValidator,
RegexValidator
from django.core.exceptions import ValidationError

```

```

from django.utils.translation import gettext_lazy as _
import datetime, time

# Generated Models

class User(models.Model):
    username = models.CharField(max_length=255, primary_key=True, null=False,
blank=False, unique=True, validators=[RegexValidator(regex=r'...',
message='Value does not match required pattern: ...')])
    age = models.IntegerField(null=True, blank=True,
validators=[MinValueValidator(8), MaxValueValidator(14)])
    bth =
models.DateField(validators=[MinValueValidator(datetime.date.fromisoformat('2
020-01-01')), MaxValueValidator(datetime.date.fromisoformat('2024-06-11'))])

    def __str__(self):
        return f'User object ({self.pk})'

```

urls.py

```

from django.urls import path
from . import views

urlpatterns = [
    path('users', views.listUsers, name='listUsers'),
    path('users/<str:x>/<str:y>', views.first_User, name='first_User'),
    path('users/sign-in', views.sign_in, name='sign_in'),
]

```

view.py

```

import json

from django.http import JsonResponse, HttpResponse, HttpResponseBadRequest

from django.views.decorators.csrf import csrf_exempt

from django.db.models import Q, Count, Sum, Avg

from .models import *

@csrf_exempt

def listUsers(request):

    if request.method != 'GET':

        return JsonResponse({'error': 'Method not allowed'}, status=405)

    return JsonResponse(list(User.objects.all().values()), safe=False)

@csrf_exempt

def first_User(request, x, y):

    if request.method != 'GET':

```



```

        return JsonResponse({'error': 'Method not allowed'}, status=405)

# Relational Logic

r_1 = User.objects.all().filter(name=y, age__lt=x)

r_2 = r_1.values('name', 'lastname')

r_3 = Person.filter()

r_4 = [{**item1, **item2} for item1 in (list(r_2) if not hasattr(r_2,
'values') else list(r_2.values())) for item2 in (list(r_4) if not
hasattr(r_4, 'values') else list(r_4.values())) if str(item1.get('name')) ==
str(item2.get('username'))]

r_5 = r_4.values('name', 'lastname', 'bth')

len_temp = (r_5.count() if hasattr(r_5, 'count') else len(r_5))

r_6 = r_5[1:len_temp:3]

res_7 = r_6

final_result = res_7

if hasattr(final_result, 'values'):

    final_result = list(final_result.values())

elif hasattr(final_result, '_meta'):

    final_result = {k: v for k, v in final_result.__dict__.items() if not
k.startswith('_')}

    return JsonResponse(final_result, safe=False)

@csrf_exempt

def sign_in(request):

    if request.method != 'POST':

        return JsonResponse({'error': 'Method not allowed'}, status=405)

    try:

        body = json.loads(request.body) if request.body else {}

    except Exception:

        return HttpResponseBadRequest('Invalid JSON')

    return JsonResponse(list(User.objects.all().values()), safe=False)

```

Conclusion

The custom DSL and compiler simplify building Django backends by automating repetitive code. Benefits include higher productivity and consistency: developers write a brief specification instead of verbose Django models and views, as supported by prior work[2][1]. The DSL makes it easier to reason about the domain (models, enums, relations) and keeps the “source of truth” at a high level[2]. However, there are trade-offs. The DSL compiler itself must be maintained, and keeping the DSL in sync with application needs can be effortful[6]. Generated code may be less optimized or more complex than custom handwritten code, since it must support general cases[7]. Debugging can also be harder if the generated output does not match expectations.

For future improvement, we would add **schema validation** and static checks in the DSL (e.g. ensuring referenced models exist, ...). Integrating with Django REST Framework (DRF) is another extension: the compiler could generate DRF serializers and viewsets instead of raw views, enabling pagination, authentication, and richer API features out-of-the-box. Other enhancements might include custom annotations for automatic admin-interface generation, permissions, or more fine-grained control over migrations. In sum, the DSL approach shows clear promise for scaffolding Django applications, and with additional features (validation, DRF support, richer type system) it could become even more powerful for real-world use. Even

Integrating with other frameworks beyond Django (like FastAPI or Flask) is straightforward due to the generic AST design.

Sources: This report references concepts and best practices from the literature on DSLs and code generation[1][2][3][6][7].

[1] Generation Technique for Django MVC Web Framework Using the Stratego Transformation Language

https://dannenkov.me/papers/MIPRO_Annenkov_Cherkashin_Model_Generation_Stratego_rewriting_language-FULL-v2.pdf

[2] [3] [6] [7] A Guide to Code Generation - Strumenta

<https://tomassetti.me/code-generation/>

[4] [5] Listeners And Visitors - Strumenta

<https://tomassetti.me/listeners-and-visitors/>